Graph Representation Models: Matrix vs Adjacency List

Graphs G = (V, E) can be represented as an Adjacency Matrix or Adjacency List.

Adjacency Matrix: A V x V 2D boolean or weight matrix where Matrix[u][v] denotes the edge between u and v. Lookup of edge existence takes O(1) time, but memory footprint is O(V^2), making it prohibitive for sparse graphs.

Adjacency List: An array or dictionary of lists where List[u] stores all adjacent vertices to u. Memory is O(V + E). Finding all neighbors of vertex u takes O(deg(u)) time, which is optimal for sparse graph traversal.

Breadth-First Search (BFS) and Depth-First Search (DFS)

Breadth-First Search (BFS) explores vertices level by level using a FIFO queue. In an unweighted graph, BFS discovers the minimum edge-count path from source s to target t in O(V + E) time.

Depth-First Search (DFS) explores as deep as possible along each branch using recursion or a LIFO stack. DFS assigns discovery and finishing timestamps used for Topological Sorting and Strongly Connected Components (Kosaraju or Tarjan). In directed graphs, back edges identify cycles.

Dijkstra's Algorithm and Min-Heap Priority Queues

Dijkstra's algorithm calculates the shortest path from a single source to all vertices in a weighted graph with non-negative edge weights.

Algorithm steps:
1. Initialize dist[source] = 0 and dist[v] = infinity for all other vertices.
2. Insert (dist[source], source) into a Min-Heap Priority Queue.
3. While priority queue is non-empty, extract vertex u with minimum tentative distance.
4. For each neighbor v with edge weight w(u, v): if dist[u] + w(u, v) < dist[v], update dist[v] = dist[u] + w(u, v) and push (dist[v], v) into the min-heap.
Using a binary min-heap, Dijkstra's algorithm executes in O((V + E) log V) time.